FULL-STACK TAKE-HOME / IMPLEMENTATION CHECKLIST

Reactiv take-home Acceptance criteria

Mobile App Home Screen Editor + Backend API | Prepared for Anthony Teixeira | 10 October 2026

43 REQUIRED	9 RECOMMENDED	4 PRODUCTION EXTENSIONS

How to use this checklist

The 43 required criteria below translate the four-page assessment into observable pass conditions. The nine Reactiv-informed recommendations and four production extensions are separate. They are engineering inferences from public projects and technical documentation, not Reactiv's private grading rubric.

Complete the required criteria first. Prioritize R01, R02, R03, R05 and R06 next because they strengthen this particular editor without expanding it into a commerce platform. Record remaining recommendations as tradeoffs if time is limited. The PDF explicitly favors a functional, clean, extensible foundation.

Each criterion has an ID, a measurable pass condition and suggested evidence. All items begin unverified; this document does not claim an implementation already passes. Screenshot evidence can demonstrate appearance; behavior and security require interaction, tests or inspection.

Interpretation choices

- Manual Save is the baseline. Imported data becomes the current draft; saving persists it. Autosave and a UI for multiple configurations are optional in the PDF.
- React Router v7 framework mode and Next.js with a server runtime are both valid. For Next.js, server-side route loading/server components fulfill the initial-load responsibility; mutations use Server Actions. For React Router, use server loaders and actions.
- REST examples below are illustrative. GraphQL operations with explicit input/output types and consistent typed errors also satisfy the assessment.
- Suggested conventions: schemaVersion 1; absolute HTTP/HTTPS image and CTA URLs; hex colors in #RGB or #RRGGBB format; portrait 3:4, landscape 4:3, square 1:1. Document any equivalent choice. These exact conventions are not specified in the PDF.
- Export/import roundtrip equality concerns schema version, section order and settings. Server-owned identity and save timestamps can differ. Importing an ID does not grant access to or choose a different server configuration.
- In-memory storage is explicitly allowed. Refresh persistence is required while the service is running; persistence across a process restart depends on the chosen storage and must be documented.

What Reactiv's real work implies

All sources were accessed on 10 October 2026. Customer statistics are Reactiv's published claims; they are not independently measured here.

Public evidence	Relevant engineering inference	Criteria
[S1] Reactiv's platform documentation describes versioned JSON configuration, ordered sections, stable entity IDs, schema validation, registered renderers and an additive extension mechanism.	Treat configuration as a durable contract. Keep rendering declarative and section identity stable.	R01-R03, R05
[S2] Med School Bro, built with Lazer, adds daily quizzes, weekly trivia and personalized recommendations to a commerce app.	A new interaction should fit through a section extension seam. Show how a quiz could fit without implementing a quiz for this assessment.	R03, R04
[S3] Hobbiesville's app supports catalog sync, custom buylist/pre-order experiences and a published 130,000-SKU catalog. Its earlier platform caused web/app inconsistencies and substantial support work.	Protect consistency and preserve edits during failures. Large catalogs motivate bounded configuration; they do not establish editor request volume.	R01, R05, R07-R09, SCL03
[S4, S5] Missouri Star and partner Domaine describe Daily Deals, shoppable tutorials, cloud-synced saved content and recovery after the previous app failed.	Preserve a simple editing journey and clear recovery paths; leave room for new content sections.	F11-F12, P03-P04, R03-R04, SCL04
[S6] Reactiv's Ecommerce API article describes server-side integration, mobile-oriented data delivery and image optimization.	Isolate service access, reserve image geometry and measure local preview responsiveness.	A10-A11, R06-R08
[S7] Reactiv's Security Overview describes merchant/store-scoped data segregation and error/event logging.	Explain the app-server trust boundary. Tenant isolation and operational diagnostics matter when moving beyond a single-configuration demo.	R09, SCL01, SCL04

The assessment is an authoring tool. Shopper counts, SKU counts and revenue cannot be converted into configuration-service requests per second. No public evidence establishes the busiest client's peak load or a numeric load target for this exercise.

Required: editor and live preview

Basis: PDF page 1, Preview Screen Components; page 2, Editor Behaviour; page 4, User Experience and graceful handling.

ID	Acceptance criterion	Evidence
F01	On first use, the app shows a documented default home screen. When a saved configuration exists, it loads that configuration and initializes both editor and preview from it. No successful load silently substitutes unrelated defaults.	First-use and saved-load walkthrough.
F02	Editing every supported field updates the preview immediately from local draft state, without Save, refresh or a successful network request. Editing one section does not change another section's settings.	Automated edit/preview test, including a delayed backend.
F03	The user can add at least two carousels, two text sections and two CTAs to the same screen. Every instance has its own editable settings and renders in the preview.	Six-section interaction test.
F04	The user can reorder sections, including moving two sections of the same type past each other. Editor, preview, export and saved reload preserve the resulting order and the settings belonging to each section. Drag-and-drop is optional; move buttons are sufficient.	Reorder test with distinct labels and colors.
F05	A carousel with multiple images scrolls horizontally and lets the user reach every image. Scrolling or changing one carousel does not affect another.	Interaction test with two multi-image carousels.
F06	Each carousel supports adding an image URL, editing an existing URL and removing an image URL. The preview displays the resulting images in the configured order. Removing the final image produces a usable empty state.	Add/edit/remove test, including the final image.
F07	Each carousel supports portrait, landscape and square views. Switching the mode changes the displayed image frame geometry and survives export/import and save/reload. The selected ratios and cropping behavior are documented.	Three-mode visual check plus persistence test.
F08	Each text section displays independently editable title and description text. Normal text, multiline descriptions and Unicode characters are retained through editing, export/import and save/reload.	Representative text fixture and roundtrip.
F09	Title and description have independently editable hex colors. Each valid value changes the corresponding preview text and survives a roundtrip; changing one color does not overwrite the other.	Separate-color test with two text sections.
F10	Each CTA displays an editable label, destination URL, button color and label color. The preview uses the configured destination and colors. The chosen navigation behavior is documented and avoids accidentally discarding an editing session.	Inspect rendered link and test each property.
F11	At representative widths of 375, 768 and 1440 CSS pixels, required controls, Save, import/export and the preview remain usable. The page has no accidental horizontal overflow; horizontal scrolling is confined to carousels. These widths are proposed verification examples.	Browser viewport review.
F12	Fields and section controls have clear labels. Temporary bad input, blank fields and empty image lists do not crash the app. Validation errors identify the affected field, and users can correct input without losing other edits. Required/optional text fields are documented.	Invalid-input and empty-state walkthrough.

Required: import, export and configuration contract

Basis: PDF page 2, Import / Export and Configuration Service; page 3, API contract and schema evolution requirements.

ID	Acceptance criterion	Evidence
D01	Export downloads parseable JSON representing the current draft, including edits since the last Save. It includes the documented schema version and preserves all sections, order, image URLs, ratios, copy, links and colors. Credentials and transient UI state are absent.	Parse an exported file and compare its payload to the draft.
D02	Importing a valid exported configuration replaces the draft and updates editor and preview together. Saving and reloading then restores the imported payload. The app's chosen import/save policy is visible and documented.	Export, alter draft, import, save and reload.
D03	Invalid JSON, an empty file or unreadable file produces a useful error. The previous draft and persisted configuration remain intact, and the user can retry with a valid file.	Malformed-file test with before/after equality.
D04	Parseable JSON with missing required fields, wrong value types, unsupported section types, invalid colors or invalid URLs is rejected gracefully. Validation completes before replacing draft state or writing data.	Parameterized semantic-invalidity tests.
D05	Every stored configuration includes id as a string, schemaVersion as a number, updatedAt as a timestamp and data as the payload. Create/update responses have the documented shape. The service owns authoritative IDs and save timestamps under the chosen contract.	Inspect API responses and stored record.
D06	The payload explicitly represents an ordered list of section types and their settings. Carousel data contains images and aspect ratio; text data contains title, description and both colors; CTA data contains label, URL and both colors. All configured settings survive a full roundtrip.	Schema inspection and full-fixture equality.
D07	Runtime validation applies to API writes as well as imports. The same accepted color/URL rules are used throughout. Under the suggested convention, #123 and #12ABEF pass; #12, #GGGGGG and script/data/file URL schemes fail. Ordinary text is rendered as text, not executable markup.	Direct API validation and safe-rendering tests.
D08	The current schema version and unknown-version policy are explicit. An unsupported future version is rejected with a clear error and no mutation. The README explains how a future migration or compatibility layer would be added; implementing migration machinery is unnecessary for version 1.	Future-version test and documentation review.

An illustrative shape is { id, schemaVersion, updatedAt, data: { sections: [{ id, type, settings }] } }. Section IDs are recommended by R02. Field names and internal nesting can differ if the contract is explicit and every required value is captured.

Required: API and private authentication

Basis: PDF page 1, Technology Requirements; page 3, API Design, Private Authentication and Server-Side API Access.

ID	Acceptance criterion	Evidence
A01	The app uses React Router v7 in framework mode or Next.js with a server runtime. State uses Context/Hooks or another idiomatic approach. TypeScript is preferred; a documented JavaScript implementation is permitted by the PDF.	Project/dependency and state-flow inspection.
A02	One protocol, REST or GraphQL, implements load-by-ID, create and update. Request/response schemas and error schemas are explicit. A UI for selecting multiple configurations is optional even though these operations are mandatory.	Contract documentation and operation tests.
A03	Loading a known ID returns that configuration. An unknown ID returns a documented not-found error instead of another configuration or unexplained defaults. For REST, GET /configurations/:id returns 200 or 404.	Known/unknown-ID requests.
A04	Creating a valid configuration stores it, assigns an ID under the documented policy and returns the created configuration. A second create does not overwrite the first. For REST, POST /configurations returns 201.	Create two records and retrieve both.
A05	Updating an existing ID stores the entire validated payload and returns an appropriate success response with updated metadata. An unknown target produces a documented not-found error. For REST, PUT /configurations/:id can return 200 or 404.	Update, retrieve and unknown-target requests.
A06	Bypassing browser validation and sending invalid API data fails with a consistent validation error. The previous record remains unchanged. REST uses an appropriate 400/422 response; GraphQL uses the chosen typed validation error.	Direct invalid-write test and stored-record comparison.
A07	Errors use a consistent documented structure with a code and safe message; field details are supplied for validation errors. Expected failures and unexpected service errors use appropriate HTTP status codes or typed GraphQL errors. Responses expose no secrets or raw internal stack traces.	Validation, not-found and injected-failure requests.
A08	Every load, create and update operation at the configuration-service boundary requires private authentication. Missing or invalid credentials fail before data is read or mutated. Hiding an endpoint or relying on CORS does not satisfy this criterion.	Exercise all three operations with no key and a wrong key.
A09	Service credentials exist only in server-side environment/configuration. They are absent from browser bundles, HTML, loader/action payloads, exports, local/session storage, browser-visible request headers and logs. Real secrets are absent from the public repository.	Server-only import review and browser/build inspection.
A10	Browser configuration reads and writes go only to the app's server layer. That layer invokes the private configuration API with server-held credentials. The actual data path exercises service authentication; calling the database directly instead of the protected API does not demonstrate the required boundary.	Browser Network inspection and authenticated service trace.

ID	Acceptance criterion	Evidence
A11	Initial configuration loading occurs in server loaders or the Next.js equivalent. Persistence occurs through server actions. Browser-side mounting effects do not replace initial server loading, and browser event handlers do not call the private service directly.	Trace initial request and Save through server modules.

Architecture boundary: browser -> app server loader/action -> privately authenticated configuration API -> storage. A separate deployable service is not explicitly required; the protected API and server-mediated data path must be real and demonstrable. Public image requests and user-activated CTA links are separate from configuration-service access and must never receive its credentials.

Required: saving, persistence and recovery

Basis: PDF pages 1-3, persistence and suggested user flow; page 4, sensible defaults and graceful error handling.

ID	Acceptance criterion	Evidence
P01	Save persists the current valid ordered payload through the required server path, creating or updating under the chosen lifecycle. Success is shown only after the configuration service acknowledges success. Saving never requires a browser-held private key.	Save walkthrough with service/storage inspection.
P02	After a successful Save, refresh restores the persisted payload and metadata through the backend. Clearing browser-local storage does not remove the saved configuration or change the server result.	Save, clear storage, reload and compare.
P03	A failed Save leaves the draft intact, shows a useful error and allows retry. It does not report success or replace the last known saved configuration with a partial write.	Inject service failure, then retry successfully.
P04	A failed initial backend load shows a recoverable error or clearly labeled fallback. It does not silently treat a service outage as a missing configuration or overwrite existing data with defaults.	Simulate unavailable service, recover and reload.
P05	The documented storage choice supports required refresh persistence. The README states restart behavior, durability, concurrency and deployment limitations. SQLite, Postgres, file-based and in-memory storage are all allowed; claimed durability is verified with a restart if applicable.	Storage inspection and documented-limitations review.
P06	A rejected or failed update leaves the previous complete configuration readable. Data and updatedAt are committed coherently under the storage approach. Validation occurs before mutation. No partially updated section list is served.	Invalid/failure write followed by retrieval.

Required: maintainability, tests and submission

Basis: PDF page 4, Technical and Code Quality Requirements, AI Usage, Submission and Evaluation Criteria.

ID	Acceptance criterion	Evidence
Q01	UI components, draft state transitions, server data access, API contract/validation and storage have clear responsibilities. Server-only modules do not enter client bundles. The reviewer can trace an edit and Save without navigating duplicated business rules.	Module and dependency review.
Q02	Meaningful automated tests exercise core behavior: editing updates the preview, repeated section types remain independent, reordering preserves settings, and carousel/text/CTA values render correctly. Tests assert observable outcomes rather than merely repeating implementation details.	Runnable tests with failure-sensitive assertions.
Q03	Environment variables configure secrets and service settings. A safe example file names required variables, and missing required server configuration causes an understandable setup error rather than an insecure fallback credential.	Fresh-environment setup and missing-variable check.
Q04	The README gives exact local setup, environment variables, app/backend start commands, test commands, a high-level architecture and notable tradeoffs/assumptions. A fresh checkout can be run using those instructions and documented dependencies.	Follow README from a clean checkout.
Q05	A genuine 1-2 page Decision Log states how AI was used, what output was changed or rejected, technical tradeoffs and improvements with more time. Significant decisions identify the proposed AI approach, the author's reasoning and verification. Entries describe actual work; no invented prompts, rejected outputs or test results.	Review log against code, commits and test evidence.
Q06	The submission provides a public repository containing the runnable source, README and Decision Log. Relevant build/type checks and tests pass under the submitted instructions. The reviewer can complete the main editing/import/export/save/reload journey without uncaught runtime errors.	Repository review, clean run and final walkthrough.

Recommended: criteria inferred from Reactiv's projects

These are proposed refinements, not additional explicit PDF requirements. Source IDs refer to the verified references below. Exact sizes, limits and timing targets here are proposed local acceptance targets, not published Reactiv requirements.

ID	Acceptance criterion	Rationale and evidence
R01	A single serializable section model drives editor state, preview, export/import and persistence. The renderer derives order and settings from the payload rather than hardcoded positional assumptions. A fixture with repeated section types renders consistently before and after a roundtrip.	[S1, S3] Configuration-driven rendering and sync. Inspect model ownership and compare fixtures.
R02	Section IDs are unique and stable through edit, reorder, export/import and save/reload; new sections receive new IDs. Rendering keys use identity rather than array positions. Duplicate section IDs in an import are rejected or normalized under an explicit policy before use.	[S1] Stable entity identity. Test moving two differently configured sections of the same type.
R03	Each section type has an identifiable schema/defaults, editor and preview renderer behind a small dispatch/registration seam. The README explains how a quiz or video section would be added without rewriting persistence or forking the editor. A simple typed switch or map is sufficient.	[S1, S2, S5] Additive customization and custom content. Code review and an extension sketch.
R04	Two contrasting brand fixtures preserve their configured colors, copy, imagery and order across editing and reload. Branding comes from configuration with sensible defaults; components do not substitute hardcoded demo brand colors. Full design-token infrastructure is optional.	[S2, S5] Brand-specific experiences. Compare both fixtures visually and structurally.
R05	Save captures draft A. Edit to B while A's save is pending, then complete A's save: preview retains B; editor still shows unsaved changes; reload restores A. Serialize saves or reconcile responses by revision/snapshot. Imported drafts are clearly distinguished from saved data.	[S1, S3] Preview/persist separation and consistency. Delay A, edit B, resolve A; verify B, dirty state, reload A.
R06	Carousel frames reserve the selected aspect ratio while images load. Broken or slow image URLs show a bounded placeholder instead of collapsing layout. Off-screen images can load lazily. A failed image does not block editing, export or Save.	[S6] Mobile media efficiency. Stub a slow image and a 404 image; inspect layout and interactions.
R07	File size, API body size, section count, image count and field lengths have documented limits applied consistently. Exceeding a limit returns an actionable error without truncation or mutation. Suggested starting limits are 1 MiB per JSON file/body, 50 sections and 20 images per carousel.	[S3, S6] Bound the authoring payload separately from catalog size. Test at and beyond each chosen limit.
R08	Record a local preview benchmark with 30 sections, including 10 carousels with 10 images each. In a production build with cached/stubbed images, measure 50 text edits; suggested target is p95 input-to-preview paint below 100 ms. Record hardware/browser, method and actual result; identify any bottleneck.	[S6] Responsive mobile experiences. A local target only, not a shopper-load or production-capacity claim.
R09	Failed load/save requests carry a safe correlation ID from app server to service logs and user-visible error details. A reviewer can locate the matching failure without logging credentials or whole configuration bodies. A console/structured logger is sufficient for the take-home.	[S3, S7] Reduce support/diagnosis cost. Inject a failure and correlate the two server layers.

The extension sketch in R03 is evidence of a clean seam, not a request to build quizzes, video commerce, loyalty, analytics or native apps before finishing the assessment.

Optional: production acceptance extensions

Use these for a short future-work discussion. They become implementation acceptance criteria only if the submission actually claims the corresponding production capability.

ID	Acceptance criterion if implemented	Evidence and scope
SCL01	Tenant scope is derived from a trusted authenticated app-server session, and service queries enforce it for load, create and update. Merchant A cannot read or write B's configuration by guessing IDs or editing browser fields. A service key authenticates a caller; it is not merchant authorization by itself.	[S7] Two-tenant isolation tests. Browser sessions may carry session credentials; private service credentials remain server-only.
SCL02	Concurrent editors use an explicit revision/ETag compare-and-set policy or another documented conflict strategy. A stale update cannot silently overwrite a newer accepted edit. With optimistic concurrency, the loser receives a recoverable 409/412 or typed conflict response, and retains its draft.	Consistency inference from [S1, S3]. Two clients save the same base revision. schemaVersion is format version, not edit revision.
SCL03	Capacity claims identify configuration size, read/write rates, concurrency, burst duration, deployment and hardware. A reproducible test reports latency/error distributions and verifies data integrity. Separate dashboard authoring from any future high-read shopper configuration delivery/cache path.	[S3, S4, S6] Define the workload first; public catalog/user figures do not prove throughput. Invalidate/version caches on writes.
SCL04	Durable storage, backup/restore, service timeouts and failure monitoring are configured under declared recovery targets. A restart and restore drill recover the last committed configuration under that policy. Any declared availability or recovery objective has measured evidence.	[S4, S7] Reliability and recoverability. A documented plan is enough if production capability is not claimed.

Other optional scope: autosave, multiple-configuration selector, section deletion/duplication, undo/redo, drag-and-drop, real Shopify integrations and native-device previews. Finish required behavior before adding these.

Verification plan

Use one fixture with two sections of each type. Give every instance different labels, colors and image URLs so identity/order mistakes are visible. Stub image responses in automated tests to keep failures attributable to the app.

Test	Scenario	Criteria covered
T01	Add all six sections; edit every required field; scroll both carousels; choose each ratio; reorder repeated types.	F02-F10, D06, Q02; R01-R02 if adopted.
T02	Export an unsaved draft; change it; import the exported file; Save; clear browser-local storage; refresh and compare the payload.	D01-D02, P01-P02, F04, F07-F10.
T03	Import malformed, empty, semantic-invalid and future-version files. Assert a useful error and unchanged draft/storage after every rejection.	D03-D04, D07-D08, F12.
T04	Load/create/update directly: valid data, unknown ID, missing credentials, wrong credentials and invalid payload. Assert typed/status errors and unchanged storage on failure.	A02-A08, D05-D07, P06.
T05	Inspect browser requests and built client assets while loading and saving. Confirm the protected service is invoked from server modules and credentials never reach the browser or export.	A09-A11, Q01, Q03.
T06	Inject failed initial load and failed Save; retain draft; retry. For R05: start saving A; edit to B; then complete A's save. Preview retains B; editor still indicates unsaved changes; reload restores A.	P03-P04; R05 and R09 if adopted.
T07	Check representative narrow/tablet/desktop widths, empty data and broken images.	F11-F12; R06 if adopted.
T08	Follow the README from a fresh checkout; run relevant checks; verify all claims and the short AI Decision Log.	P05, Q03-Q06.

Automate the core editor/preview scenarios required by Q02. API/authentication, roundtrip and failure tests provide strong additional evidence. A few focused integration tests are more useful than a superficial coverage percentage.

Suggested review order

1. Prove the required workflow: edit -> preview -> export/import -> Save -> refresh.
2. Prove the security boundary with direct unauthorized service requests and browser/build inspection.
3. Demonstrate repeated-section identity and invalid-input/failure recovery.
4. Explain the small configuration model, extension seam and storage tradeoff.
5. Use the genuine Decision Log to explain one accepted AI output and one changed/rejected output, including how each was checked. If no output was rejected, describe an actual change rather than inventing a rejection.

PDF requirement traceability

Assessment requirement	Page	Acceptance IDs
Framework, language, protocol and state approach	1	A01-A02
Carousel, text and CTA fields and display	1-2	F05-F10, D06-D07
Live preview, multiple sections, ordering and responsive UX	1-2	F01-F04, F11-F12
JSON import/export and invalid files	2	D01-D04, D08
Configuration metadata, payload and storage tradeoffs	2	D05-D06, P05-P06
Load/create/update, contracts, validation, errors, schema evolution	2-3	A02-A07, D07-D08
Private authentication and server-only credentials	3	A08-A09, Q03
Server loading/actions and refresh persistence	3	A10-A11, P01-P04
Separation of concerns, core automated tests and graceful handling	4	Q01-Q03, F12, P03-P04
AI usage, Decision Log, README, public repository and evaluation	4	Q04-Q06; inferred refinements support extensibility/pragmatism.

References

- Assessment: Reactiv - Take-Home Challenge - Full Stack - Build a Mobile App Home Screen Editor + Backend API.pdf, supplied four-page PDF. Page references above use the PDF's page order.

- [S1] Reactiv, How Reactiv Works. Technical documentation; last updated 3 September 2026. https://docs.reactivapp.com/developers/platform-overview

- [S2] Reactiv, Med School Bro customer story. Daily quizzes/trivia and custom learning-to-commerce experience built with Lazer. https://www.reactiv.ai/customer-stories/med-school-bro

- [S3] Reactiv, Hobbiesville customer story. Customization, sync, catalog scale and support burden. https://www.reactiv.ai/customer-stories/hobbiesville

- [S4] Reactiv, Missouri Star Quilt Co. customer story. Custom content, cloud sync and replacement of a failed app. https://www.reactiv.ai/customer-stories/missouri-star-quilt-co

- [S5] Reactiv, Domaine customer story. Agency extension model and Missouri Star's shoppable tutorials/Daily Deals. https://www.reactiv.ai/customer-stories/domaine

- [S6] Reactiv, Reactiv's Ecommerce API is leading the way to a new future for mobile commerce. Engineering article, 19 November 2024. https://www.reactiv.ai/post/reactiv-ecommerce-api

- [S7] Reactiv, Security Overview. Technical documentation; last updated 9 June 2026. https://docs.reactivapp.com/trust/security

Evidence boundary: these are first-party public descriptions and technical docs, not access to Reactiv's private code or interview rubric. The test procedures, implementation suggestions and numeric local targets are authored for this assessment.